Practical 3 – I Miss My Cats

250038453

Overview

Cat Game is a 2D platformer that centers around the goal of helping a trio of cats, Edgar, Alan, and Poe, find their food. In this rendition, I added level generation and difficulty scaling. Mechanically, the player can jump from platform to platform, slowly making their way up the level, until they find the food wl. The player can also switch between the cats to use different special abilities, defense, climbing, and better jumping, that help the player navigate through the level. Enemies also navigate through the level with the goal of killing the player and interact with the physics system similar to the player.

Design

In this iteration of Cat Game, I added constraint-based procedural generation of the tile map, Perlin noise generation of platform terrain, procedural spawning of other entities (enemies, special platforms, start and end), and difficulty scaling for the generation.

First the tile map itself is generated through a constraint-based system based on how far the player is able to move from platform to platform. It generates a platform within a reachable range of the player and then generates the next platform. Additionally, these platforms have terrain that is generated by Perlin noise to give them slight variation. It is important to note that each platform is independent, so random (and sometimes biased based on difficulty) generations are used to determine how platforms are placed compared to the previous one.

Beyond platforms and based off them, other tiles of the tile map are generated. Enemies can only spawn in places where the player exists in the level and they cannot spawn on the starting platform (for peace of mind). The start and end of the level are guaranteed to be on the first and last (generally highest most) platform respectively, to guarantee use of the entire level.

Finally, difficulty scaling affects all the above content (except player spawn and end positions). For enemies, a higher difficulty means the more enemies are likely to spawn, and sometimes more than one can spawn at a time. In terms of platform generation, a higher difficulty means that platforms are more likely to spawn further away from each other, so they are more difficult to reach. The combination of these generations makes a significant difference on the difficulty of the level.

Technical Description

The technical challenges in this iteration of Cat Game can be separated into constraint and noise-based platform generation, the procedural generation of other tiles, and difficulty scaling

In terms of the platforms, platforms were generated in a recursive train based off the previous platform. Each platform is guaranteed to be within a logically reachable distance by the player through the definition of movement limits (like how far the player can jump vertically and horizontally). Each platform determined the offset (jump distance) it would spawn away from the platform based off random number generation with bias (explained later in difficulty) and the above-mentioned constraints. Additionally, platforms originally spawned on top of each other because there was no constraint on overlaps and usable player space. To remedy this, I added a new number to the tile map file that represents open player space. When the tile map is generated, these tiles are generated as air, but in the generation algorithm they are used to determine if the generation of a platform intersects with playable space. If a platform intersects with it, then a new platform is generated that fits in the space. Essentially, the algorithm only spawns platforms that are within reach of the player. Also, each platform uses Perlin noise to generate terrain. The platforms are not very large, so I added a significant and a randomly adjusting increment that makes sure platforms have terrain of some variety. Additionally, variety between platforms was important to me during development, so I set the initial noise factor and the increment as random numbers (within a specified range) to help prioritize the individuality of platforms.

Beyond platforms, all other entities were spawned based on where platforms were placed. The start of the game is simply spawned on the starting platform at the bottom left corner of the map. Additionally, the losing tiles (triggered by falling off the platforms) cover the entire bottom of the tile map. After that the spawning gets a bit more complicated. The win condition (food bowl) is spawned based off the recursive platform spawning algorithm. Each iteration of the algorithm returns the result of the generation of its child (or its parent if generation fails), and when the height limit is reached, the algorithm returns the final platform. The food bowl is placed on top of this platform, guaranteeing that the end is both the highest and hardest to reach platform. This makes the player navigate as close to the entire level as possible. Finally, enemies are generated based off the player space generated in the above section in addition to enemy spawning space (that can be overwritten) above it. When each platform is generated, an enemy can spawn on it. The chance that an enemy spawns is initially almost 0, so enemies don't spawn on the initial platforms. However, the chance of enemies spawning grows as platforms are generated without enemies, so at some point an enemy is forced to spawn. Platforms are restricted to only spawning 1 enemy unless on the highest difficulty. When enemies spawn, the spawn chance variable is reset and the cycle starts again. I based this spawning off on how tunneling worked in lectures on Procedural Generation in class. Initially I had enemies spawning in a separate function based exclusively on player space and not platforms, but this ended up creating weird clusters of enemies. I changed it to a platform-based system for a cleaner and realistically spread-out approach.

Finally, difficulty scaling was one of the final and most impactful things I added. It adjusts how all the above generations behave except for the player spawn point, the end point,

and the death zones. For platforms, a higher difficulty maxes out horizontal and vertical ranges for the distance between platforms and makes more difficult jumps more likely to spawn. This is accomplished through bias. Essentially a higher difficulty makes the random number, that determines the height and distance respectively, bias toward one end of the range by modifying it with a power function after generation. Since these numbers are always generated between 1 and zero exclusively, the bias cannot force a platform to be generated outside of a reachable area. Of course, the opposite is true for easier difficulties that bias towards easier distances. Additionally, difficulty scaling also modifies how enemies spawn. The higher the difficulty, the more likely enemies are to spawn. This is accomplished by mapping the starting enemy spawn chance and increment with the difficulty. When difficulty is higher, the spawn chance and increment follow. Additionally, on the highest difficulty, the restriction of one enemy per platform is entirely ignored, so that an annoying number of enemies can spawn. However, on the easiest difficulty, no enemies spawn. Which makes it incredibly difficult for the player to lose.

Critical Review

Overall, I feel like I did a good job with this practical by implementing some styles of procedural content generation that were introduced in class. Due to the nature of creating a 2D platformer, I had to do further research on these generation techniques to help figure out how they would be applicable to what I was doing. Overall, I think my project suffered the most from a lack of planning. I did not come up with a structure outline before beginning my project this time, so I subsequently spent a lot of time debugging and building the algorithm brick by brick. A plan would've not only saved me time but also allowed me to explore more styles of generation and have a cleaner product.

Despite that, I created a procedural generation algorithm that effectively generates levels of my games. This algorithm works consistently and increases playability of the game through the customization of user experience. In all honesty, I got a bit distracted trying to beat my game on the hardest difficulty and a large map. It is not easy. I think the difference in difficulty is meaningful and noticeable. Beyond that, I finally created a menu for my game, which gives it a cleaner appearance.

Conclusion

Cat Game could have a future beyond the timeline of this course, and I may continue working on it for fun and practice various techniques. I went into this practical expecting the addition of procedural generation to be easier than the previous practicals, but I was proven wrong. There are many different things to consider when creating an algorithm that generates levels, and the intricacies of each thing and the interactions between them make this development difficult. Procedural generation cannot be an afterthought, and the quality of an algorithm can make or break a player's experience.

References

Processing Documentation. https://processing.org/reference

Specifically, Noise. https://processing.org/reference/noise_.html

Explanation of Perlin Noise. https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/perlin-noise-part-2/perlin-noise.html

Examples with Terrain (Noise). https://code2d.wordpress.com/2020/07/21/perlin-noise/

Lecture Materials (L12 and L13). https://studres.cs.st-andrews.ac.uk/CS4303/Lectures/